

Câu 1. Nạp dữ liệu và mô tả cấu trúc (5 điểm)

```
In [22]: #1
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn

from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing(as_frame=True)
df = housing.frame.copy()
```

```
In [21]: #2
print(df.shape)

n_rows, n_cols = df.shape
print("Hàng:", n_rows)
print("Cột:", n_cols)

#features và target
features = housing.feature_names
target_col = "MedHouseVal"

print("Số cột feature:", len(features))
print("feature:", features)
print("Số cột target:", 1)
print("target:", target_col)

#Kiểm chứng
assert len(features) + 1 == n_cols, "check lại"
print("{} feature + 1 target = {} cột".format(len(features), n_cols))
```

```
(20640, 9)
Hàng: 20640
Cột: 9
Số cột feature: 8
feature: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']
Số cột target: 1
target: MedHouseVal
8 feature + 1 target = 9 cột
```

- data có 20640 hàng, 9 cột, 8 features, 1 target
- Target: MedHouseVal

```
In [23]: #3
print(df.head)
print(df.tail)
```

```
<bound method NDFrame.head of
tude \
0      8.3252      41.0  6.984127  1.023810      322.0  2.555556      37.88
1      8.3014      21.0  6.238137  0.971880      2401.0  2.109842      37.86
2      7.2574      52.0  8.288136  1.073446      496.0  2.802260      37.85
3      5.6431      52.0  5.817352  1.073059      558.0  2.547945      37.85
4      3.8462      52.0  6.281853  1.081081      565.0  2.181467      37.85
...      ...      ...      ...      ...      ...      ...
20635  1.5603      25.0  5.045455  1.133333      845.0  2.560606      39.48
20636  2.5568      18.0  6.114035  1.315789      356.0  3.122807      39.49
20637  1.7000      17.0  5.205543  1.120092      1007.0  2.325635      39.43
20638  1.8672      18.0  5.329513  1.171920      741.0  2.123209      39.43
20639  2.3886      16.0  5.254717  1.162264      1387.0  2.616981      39.37
```

```
Longitude  MedHouseVal
0      -122.23      4.526
1      -122.22      3.585
2      -122.24      3.521
3      -122.25      3.413
4      -122.25      3.422
...      ...      ...
20635  -121.09      0.781
20636  -121.21      0.771
20637  -121.22      0.923
20638  -121.32      0.847
20639  -121.24      0.894
```

```
[20640 rows x 9 columns]>
```

```
<bound method NDFrame.tail of
tude \
0      8.3252      41.0  6.984127  1.023810      322.0  2.555556      37.88
1      8.3014      21.0  6.238137  0.971880      2401.0  2.109842      37.86
2      7.2574      52.0  8.288136  1.073446      496.0  2.802260      37.85
3      5.6431      52.0  5.817352  1.073059      558.0  2.547945      37.85
4      3.8462      52.0  6.281853  1.081081      565.0  2.181467      37.85
...      ...      ...      ...      ...      ...      ...
20635  1.5603      25.0  5.045455  1.133333      845.0  2.560606      39.48
20636  2.5568      18.0  6.114035  1.315789      356.0  3.122807      39.49
20637  1.7000      17.0  5.205543  1.120092      1007.0  2.325635      39.43
20638  1.8672      18.0  5.329513  1.171920      741.0  2.123209      39.43
20639  2.3886      16.0  5.254717  1.162264      1387.0  2.616981      39.37
```

```
Longitude  MedHouseVal
0      -122.23      4.526
1      -122.22      3.585
2      -122.24      3.521
3      -122.25      3.413
4      -122.25      3.422
...      ...      ...
20635  -121.09      0.781
20636  -121.21      0.771
20637  -121.22      0.923
20638  -121.32      0.847
20639  -121.24      0.894
```

```
[20640 rows x 9 columns]>
```

```
In [24]: #4
print(df.info)
```

```
<bound method DataFrame.info of
titude \
0      8.3252      41.0  6.984127  1.023810      322.0  2.555556      37.88
1      8.3014      21.0  6.238137  0.971880      2401.0  2.109842      37.86
2      7.2574      52.0  8.288136  1.073446      496.0  2.802260      37.85
3      5.6431      52.0  5.817352  1.073059      558.0  2.547945      37.85
4      3.8462      52.0  6.281853  1.081081      565.0  2.181467      37.85
...      ...      ...      ...      ...      ...      ...
20635  1.5603      25.0  5.045455  1.133333      845.0  2.560606      39.48
20636  2.5568      18.0  6.114035  1.315789      356.0  3.122807      39.49
20637  1.7000      17.0  5.205543  1.120092      1007.0  2.325635      39.43
20638  1.8672      18.0  5.329513  1.171920      741.0  2.123209      39.43
20639  2.3886      16.0  5.254717  1.162264      1387.0  2.616981      39.37

Longitude  MedHouseVal
0      -122.23      4.526
1      -122.22      3.585
2      -122.24      3.521
3      -122.25      3.413
4      -122.25      3.422
...      ...      ...
20635  -121.09      0.781
20636  -121.21      0.771
20637  -121.22      0.923
20638  -121.32      0.847
20639  -121.24      0.894
```

[20640 rows x 9 columns]>

In [26]: #5

```
desc = df.describe().T
print(desc)
```

	count	mean	std	min	25%	\
MedInc	20640.0	3.870671	1.899822	0.499900	2.563400	
HouseAge	20640.0	28.639486	12.585558	1.000000	18.000000	
AveRooms	20640.0	5.429000	2.474173	0.846154	4.440716	
AveBedrms	20640.0	1.096675	0.473911	0.333333	1.006079	
Population	20640.0	1425.476744	1132.462122	3.000000	787.000000	
AveOccup	20640.0	3.070655	10.386050	0.692308	2.429741	
Latitude	20640.0	35.631861	2.135952	32.540000	33.930000	
Longitude	20640.0	-119.569704	2.003532	-124.350000	-121.800000	
MedHouseVal	20640.0	2.068558	1.153956	0.149990	1.196000	

	50%	75%	max
MedInc	3.534800	4.743250	15.000100
HouseAge	29.000000	37.000000	52.000000
AveRooms	5.229129	6.052381	141.909091
AveBedrms	1.048780	1.099526	34.066667
Population	1166.000000	1725.000000	35682.000000
AveOccup	2.818116	3.282261	1243.333333
Latitude	34.260000	37.710000	41.950000
Longitude	-118.490000	-118.010000	-114.310000
MedHouseVal	1.797000	2.647250	5.000010

- Count: So sánh count giữa các cột
- mean Giá trị trung bình
- std Độ lệch chuẩn — đo độ phân tán của dữ liệu quanh mean
- min Giá trị nhỏ nhất
- 25% Tứ phân vị thứ nhất (Q1) — 25% dữ liệu nhỏ hơn giá trị này
- 50% Trung vị (median) — Q2
- 75% Tứ phân vị thứ ba (Q3) — 75% dữ liệu nhỏ hơn giá trị này
- max Giá trị lớn nhất

In [28]: #6

```
print(df.shape)

print(df.head(10))

print(df.tail(10))

df.info()
```

```

print(df.describe().T)

print(df.columns.tolist())

target_col = "MedHouseVal"
features = [c for c in df.columns if c != target_col]
print("\nBiến mục tiêu (target):", target_col)
print("Số lượng feature:", len(features))
print("Danh sách feature:", features)

print(housing.DESCR)

```

(20640, 9)

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	
3	5.6431	52.0	5.817352	1.073059	558.0	2.547945	37.85	
4	3.8462	52.0	6.281853	1.081081	565.0	2.181467	37.85	
5	4.0368	52.0	4.761658	1.103627	413.0	2.139896	37.85	
6	3.6591	52.0	4.931907	0.951362	1094.0	2.128405	37.84	
7	3.1200	52.0	4.797527	1.061824	1157.0	1.788253	37.84	
8	2.0804	42.0	4.294118	1.117647	1206.0	2.026891	37.84	
9	3.6912	52.0	4.970588	0.990196	1551.0	2.172269	37.84	

	Longitude	MedHouseVal
0	-122.23	4.526
1	-122.22	3.585
2	-122.24	3.521
3	-122.25	3.413
4	-122.25	3.422
5	-122.25	2.697
6	-122.25	2.992
7	-122.25	2.414
8	-122.26	2.267
9	-122.25	2.611

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	\
20630	3.5673	11.0	5.932584	1.134831	1257.0	2.824719	39.29	
20631	3.5179	15.0	6.145833	1.141204	1200.0	2.777778	39.33	
20632	3.1250	15.0	6.023377	1.080519	1047.0	2.719481	39.26	
20633	2.5495	27.0	5.445026	1.078534	1082.0	2.832461	39.19	
20634	3.7125	28.0	6.779070	1.148256	1041.0	3.026163	39.27	
20635	1.5603	25.0	5.045455	1.133333	845.0	2.560606	39.48	
20636	2.5568	18.0	6.114035	1.315789	356.0	3.122807	39.49	
20637	1.7000	17.0	5.205543	1.120092	1007.0	2.325635	39.43	
20638	1.8672	18.0	5.329513	1.171920	741.0	2.123209	39.43	
20639	2.3886	16.0	5.254717	1.162264	1387.0	2.616981	39.37	

	Longitude	MedHouseVal
20630	-121.32	1.120
20631	-121.40	1.072
20632	-121.45	1.156
20633	-121.53	0.983
20634	-121.56	1.168
20635	-121.09	0.781
20636	-121.21	0.771
20637	-121.22	0.923
20638	-121.32	0.847
20639	-121.24	0.894

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 20640 entries, 0 to 20639

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	MedInc	20640 non-null	float64
1	HouseAge	20640 non-null	float64
2	AveRooms	20640 non-null	float64
3	AveBedrms	20640 non-null	float64
4	Population	20640 non-null	float64
5	AveOccup	20640 non-null	float64
6	Latitude	20640 non-null	float64
7	Longitude	20640 non-null	float64
8	MedHouseVal	20640 non-null	float64

dtypes: float64(9)

memory usage: 1.4 MB

	count	mean	std	min	25%	\
MedInc	20640.0	3.870671	1.899822	0.499900	2.563400	

HouseAge	20640.0	28.639486	12.585558	1.000000	18.000000
AveRooms	20640.0	5.429000	2.474173	0.846154	4.440716
AveBedrms	20640.0	1.096675	0.473911	0.333333	1.006079
Population	20640.0	1425.476744	1132.462122	3.000000	787.000000
AveOccup	20640.0	3.070655	10.386050	0.692308	2.429741
Latitude	20640.0	35.631861	2.135952	32.540000	33.930000
Longitude	20640.0	-119.569704	2.003532	-124.350000	-121.800000
MedHouseVal	20640.0	2.068558	1.153956	0.149990	1.196000

	50%	75%	max
MedInc	3.534800	4.743250	15.000100
HouseAge	29.000000	37.000000	52.000000
AveRooms	5.229129	6.052381	141.909091
AveBedrms	1.048780	1.099526	34.066667
Population	1166.000000	1725.000000	35682.000000
AveOccup	2.818116	3.282261	1243.333333
Latitude	34.260000	37.710000	41.950000
Longitude	-118.490000	-118.010000	-114.310000
MedHouseVal	1.797000	2.647250	5.000010

['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude', 'MedHouseVal']

Biến mục tiêu (target): MedHouseVal

Số lượng feature: 8

Danh sách feature: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup', 'Latitude', 'Longitude']

.. _california_housing_dataset:

California Housing dataset

****Data Set Characteristics:****

:Number of Instances: 20640

:Number of Attributes: 8 numeric, predictive attributes and the target

:Attribute Information:

- MedInc median income in block group
- HouseAge median house age in block group
- AveRooms average number of rooms per household
- AveBedrms average number of bedrooms per household
- Population block group population
- AveOccup average number of household members
- Latitude block group latitude
- Longitude block group longitude

:Missing Attribute Values: None

This dataset was obtained from the StatLib repository.

https://www.dcc.fc.up.pt/~ltorgo/Regression/cal_housing.html

The target variable is the median house value for California districts, expressed in hundreds of thousands of dollars (\$100,000).

This dataset was derived from the 1990 U.S. census, using one row per census block group. A block group is the smallest geographical unit for which the U.S. Census Bureau publishes sample data (a block group typically has a population of 600 to 3,000 people).

A household is a group of people residing within a home. Since the average number of rooms and bedrooms in this dataset are provided per household, these columns may take surprisingly large values for block groups with few households and many empty houses, such as vacation resorts.

It can be downloaded/loaded using the

:func:`sklearn.datasets.fetch\_california\_housing` function.

.. rubric:: References

- Pace, R. Kelley and Ronald Barry, Sparse Spatial Autoregressions, Statistics and Probability Letters, 33 (1997) 291-297

Mô tả:

Bộ dữ liệu California Housing gồm 20604 hàng, 9 cột, trong đó có 8 thuộc tính đầu vào (feature) và 1 biến mục tiêu (target) là MedHouseVal — giá nhà trung vị tính theo đơn vị 100.000 USD. Các feature bao gồm: MedInc (thu nhập trung vị khu vực), HouseAge (tuổi trung vị của nhà), AveRooms và AveBedrms (số phòng/phòng ngủ trung bình mỗi hộ), Population (dân số khu vực), AveOccup (số người ở trung bình mỗi hộ), Latitude và Longitude (tọa độ địa lý). Kết quả info() cho thấy kiểu dữ liệu chủ yếu là float64 ở tất cả các cột, và không có giá trị null (non-null count bằng đúng số dòng). Qua describe().T, feature AveRooms, AveBedrms, AveOccup, Population có giá trị max lớn bất thường so với mean và Q3 — cho thấy dấu hiệu outlier rõ rệt. Biến target MedHouseVal cũng có dấu hiệu bị capped (giới hạn trần) ở giá trị tối đa, một đặc điểm đã biết của dataset này (giá nhà trên \$500,000 bị gộp về mức trần khi thu thập).

Câu 2. EDA - khám phá và trực quan hóa dữ liệu (10 điểm)

In [29]: #7

```
desc = df.describe().T
desc['range'] = desc['max'] - desc['min']
desc['cv'] = desc['std'] / desc['mean']
desc['skewness'] = df.skew()

print(desc[['mean', 'std', 'min', 'max', 'range', 'cv', 'skewness']])
print(desc['cv'].sort_values(ascending=False))
print(desc['skewness'].abs().sort_values(ascending=False))
```

	mean	std	min	max	range \
MedInc	3.870671	1.899822	0.499900	15.000100	14.500200
HouseAge	28.639486	12.585558	1.000000	52.000000	51.000000
AveRooms	5.429000	2.474173	0.846154	141.909091	141.062937
AveBedrms	1.096675	0.473911	0.333333	34.066667	33.733333
Population	1425.476744	1132.462122	3.000000	35682.000000	35679.000000
AveOccup	3.070655	10.386050	0.692308	1243.333333	1242.641026
Latitude	35.631861	2.135952	32.540000	41.950000	9.410000
Longitude	-119.569704	2.003532	-124.350000	-114.310000	10.040000
MedHouseVal	2.068558	1.153956	0.149990	5.000010	4.850020

	cv	skewness
MedInc	0.490825	1.646657
HouseAge	0.439448	0.060331
AveRooms	0.455733	20.697869
AveBedrms	0.432134	31.316956
Population	0.794444	4.935858
AveOccup	3.382356	97.639561
Latitude	0.059945	0.465953
Longitude	-0.016756	-0.297801
MedHouseVal	0.557855	0.977763

AveOccup	3.382356
Population	0.794444
MedHouseVal	0.557855
MedInc	0.490825
AveRooms	0.455733
HouseAge	0.439448
AveBedrms	0.432134
Latitude	0.059945
Longitude	-0.016756
Name: cv, dtype: float64	
AveOccup	97.639561
AveBedrms	31.316956
AveRooms	20.697869
Population	4.935858
MedInc	1.646657
MedHouseVal	0.977763
Latitude	0.465953
Longitude	0.297801
HouseAge	0.060331
Name: skewness, dtype: float64	

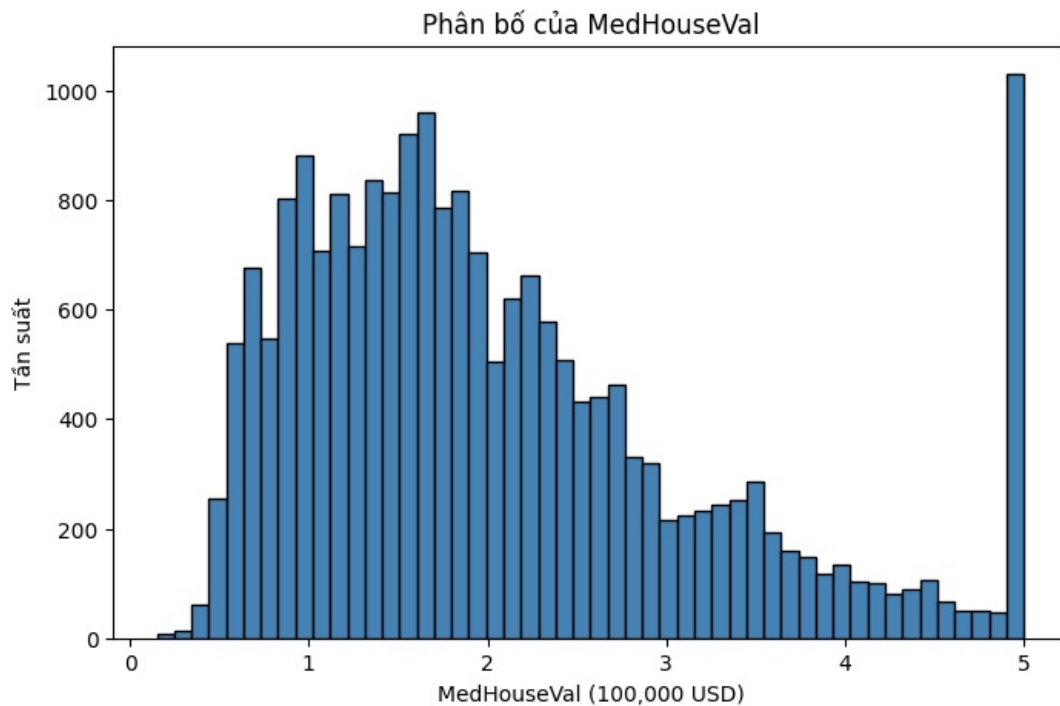
Nhận xét:

- 3 biến có độ phân tán lớn nhất hoặc phân bố lệch: AveOccup, AveBedrms, AveRooms

In [30]: #8

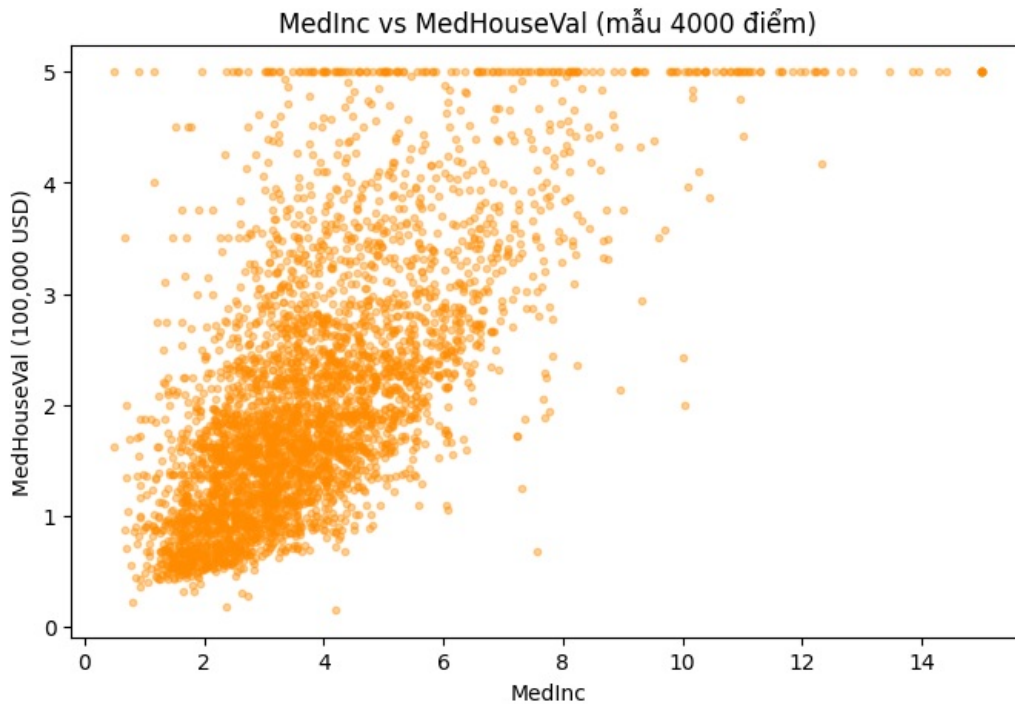
```
plt.figure(figsize=(8, 5))
```

```
plt.hist(df['MedHouseVal'], bins=50, color='steelblue', edgecolor='black')
plt.xlabel('MedHouseVal (100,000 USD)')
plt.ylabel('Tần suất')
plt.title('Phân bố của MedHouseVal')
plt.show()
```



- Nhận xét: Phân bố lệch phải (right-skewed): phần lớn giá trị tập trung trong khoảng 1.0 – 2.0, đỉnh (mode) rơi vào khoảng 1.5 – 1.6, khớp với skewness = 0.98 đã tính trước đó. Cột bất thường tại giá trị 5.0: có khoảng hơn 1000 mẫu dồn vào đúng mức giá trị max = 5.0 — đây chính là bằng chứng trực quan cho việc dữ liệu bị capped (giới hạn trần khi thu thập): mọi khu vực có giá nhà trung vị thực tế $\geq$ \$500,000 đều bị gán về đúng 5.00001. Đây không phải là một cụm dữ liệu tự nhiên mà là artefact cần lưu ý khi xây mô hình (có thể cân nhắc loại bỏ nhóm này hoặc xử lý riêng). Có một vùng trũng nhẹ quanh 2.5 – 3.0 trước khi giảm dần, phân bố không hoàn toàn trơn (unimodal đơn giản).

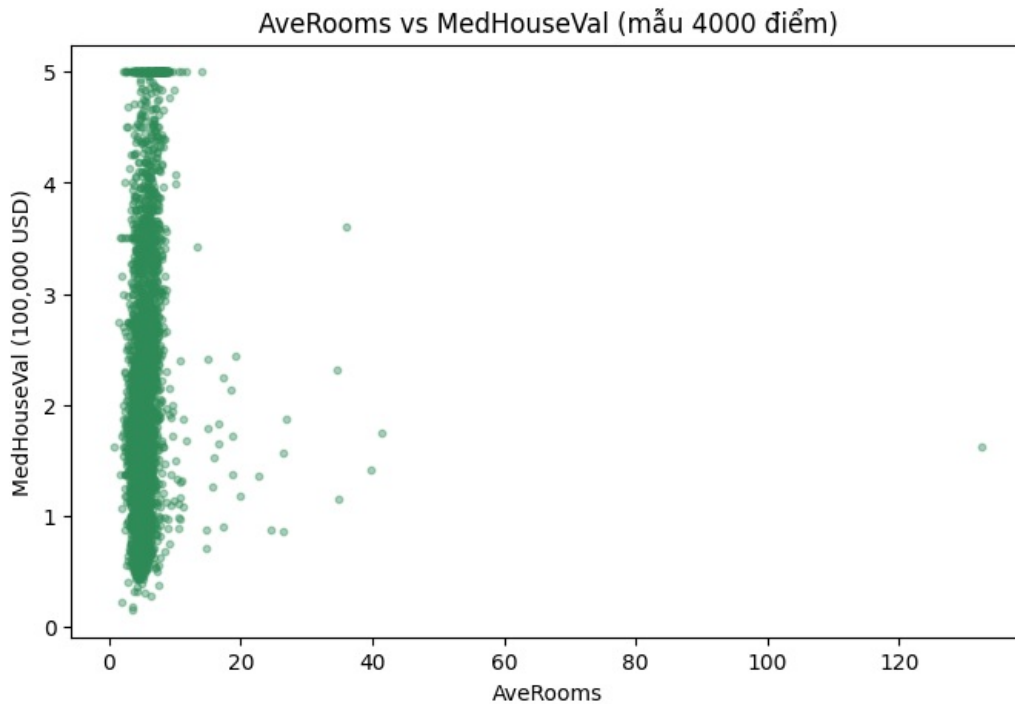
```
In [32]: #9
sample = df.sample(n=4000, random_state=42)
plt.figure(figsize=(8, 5))
plt.scatter(sample['MedInc'], sample['MedHouseVal'], alpha=0.4, s=10, color='darkorange')
plt.xlabel('MedInc')
plt.ylabel('MedHouseVal (100,000 USD)')
plt.title('MedInc vs MedHouseVal (mẫu 4000 điểm)')
plt.show()
```



- Nhận xét: Tương quan dương rõ rệt: khi MedInc tăng, MedHouseVal có xu hướng tăng theo — hợp lý về mặt kinh tế (khu vực thu nhập cao thường có giá nhà cao). Dải điểm nằm ngang ở đúng mức 5.0: xác nhận lại hiệu ứng capping đã thấy ở histogram — nhiều điểm có MedInc khác nhau (từ 2 đến 15) nhưng đều bị "kéo phẳng" về MedHouseVal = 5.0, làm nhiễu quan hệ tuyến tính thực sự giữa 2 biến. Phương sai tăng dần theo MedInc (heteroscedasticity): ở vùng MedInc thấp, các điểm co cụm chặt gần đường xu hướng; khi MedInc tăng, độ phân tán của MedHouseVal cũng tăng — dấu hiệu cần lưu ý nếu dùng hồi quy tuyến tính (giả định phương sai đồng nhất có thể bị vi phạm). Một vài điểm ngoại lai với MedInc rất cao (>12) nhưng MedHouseVal không tương ứng cao — số lượng ít, không ảnh hưởng nhiều đến xu hướng chung.

```
In [33]: #10
sample10 = df.sample(n=4000, random_state=42)

plt.figure(figsize=(8, 5))
plt.scatter(sample10['AveRooms'], sample10['MedHouseVal'], alpha=0.4, s=10, color='seagreen')
plt.xlabel('AveRooms')
plt.ylabel('MedHouseVal (100,000 USD)')
plt.title('AveRooms vs MedHouseVal (mẫu 4000 điểm)')
plt.show()
```

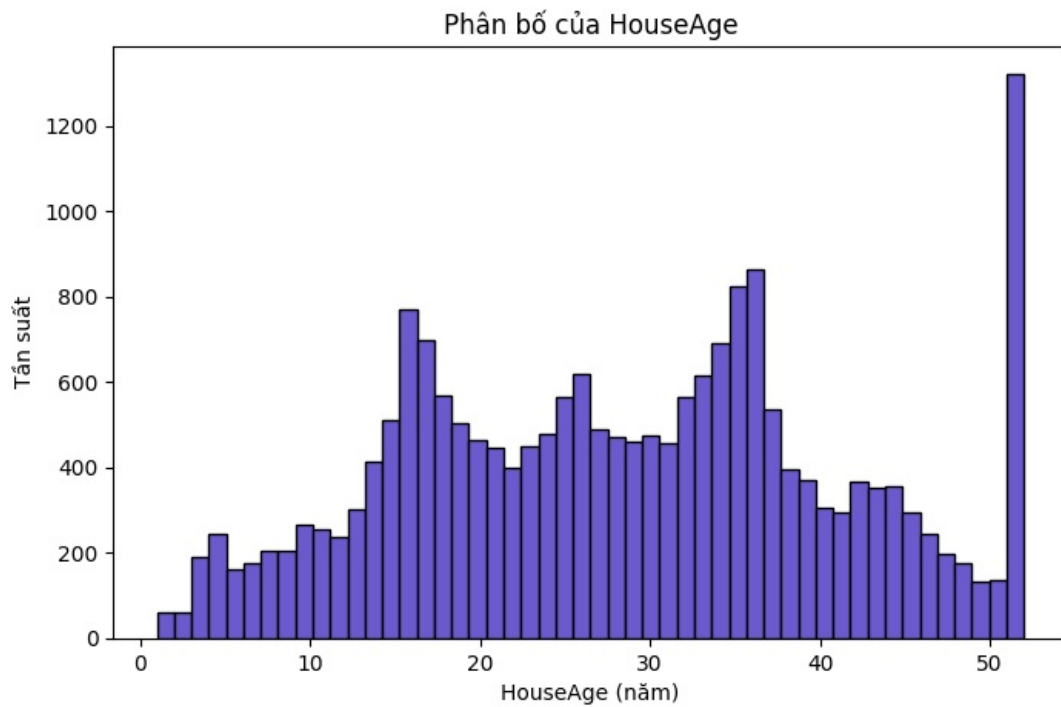


Nhận xét: Phần lớn dữ liệu tập trung trong khoảng AveRooms từ 2–10, không cho thấy xu hướng tuyến tính rõ ràng với MedHouseVal trong vùng này. Có một điểm ngoại lai rất rõ tại AveRooms ≈ 132 nhưng MedHouseVal chỉ khoảng 1.6 — hoàn toàn lệch khỏi xu hướng chung, xác nhận đây là dữ liệu bất thường (có thể là khu vực đặc biệt như ký túc xá/nhà tập thể). Một vài điểm khác ở AveRooms 20–40 cũng nằm rải rác, không theo mẫu hình rõ ràng, càng củng cố nhận định AveRooms không phải là biến dự đoán mạnh cho giá nhà khi đứng riêng lẻ.

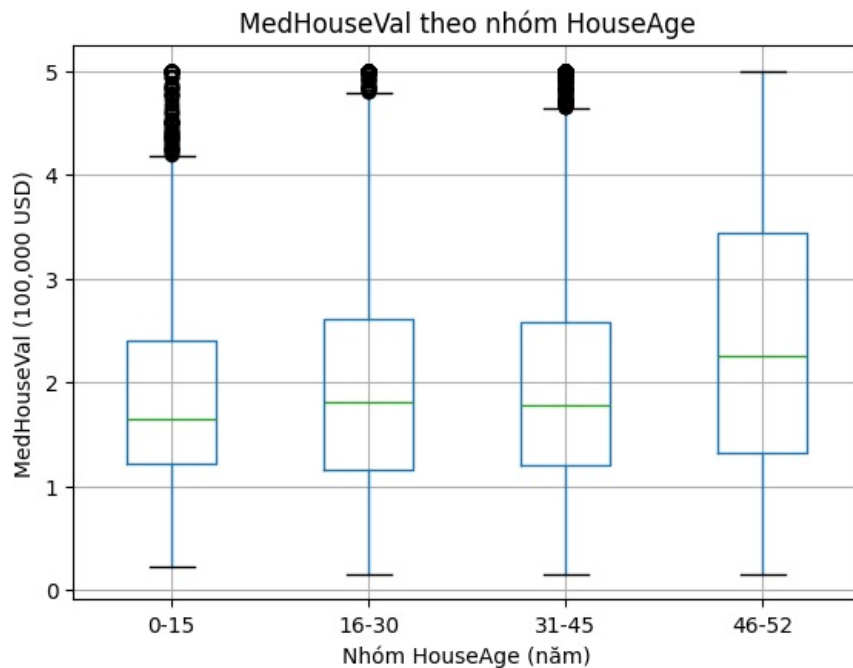
```
In [34]: #11
plt.figure(figsize=(8, 5))
plt.hist(df['HouseAge'], bins=50, color='slateblue', edgecolor='black')
plt.xlabel('HouseAge (năm)')
plt.ylabel('Tần suất')
plt.title('Phân bố của HouseAge')
plt.show()

df['HouseAge_bin'] = pd.cut(df['HouseAge'], bins=[0, 15, 30, 45, 52],
                             labels=['0-15', '16-30', '31-45', '46-52'])

plt.figure(figsize=(8, 5))
df.boxplot(column='MedHouseVal', by='HouseAge_bin')
plt.xlabel('Nhóm HouseAge (năm)')
plt.ylabel('MedHouseVal (100,000 USD)')
plt.title('MedHouseVal theo nhóm HouseAge')
plt.suptitle('')
plt.show()
```



<Figure size 800x500 with 0 Axes>

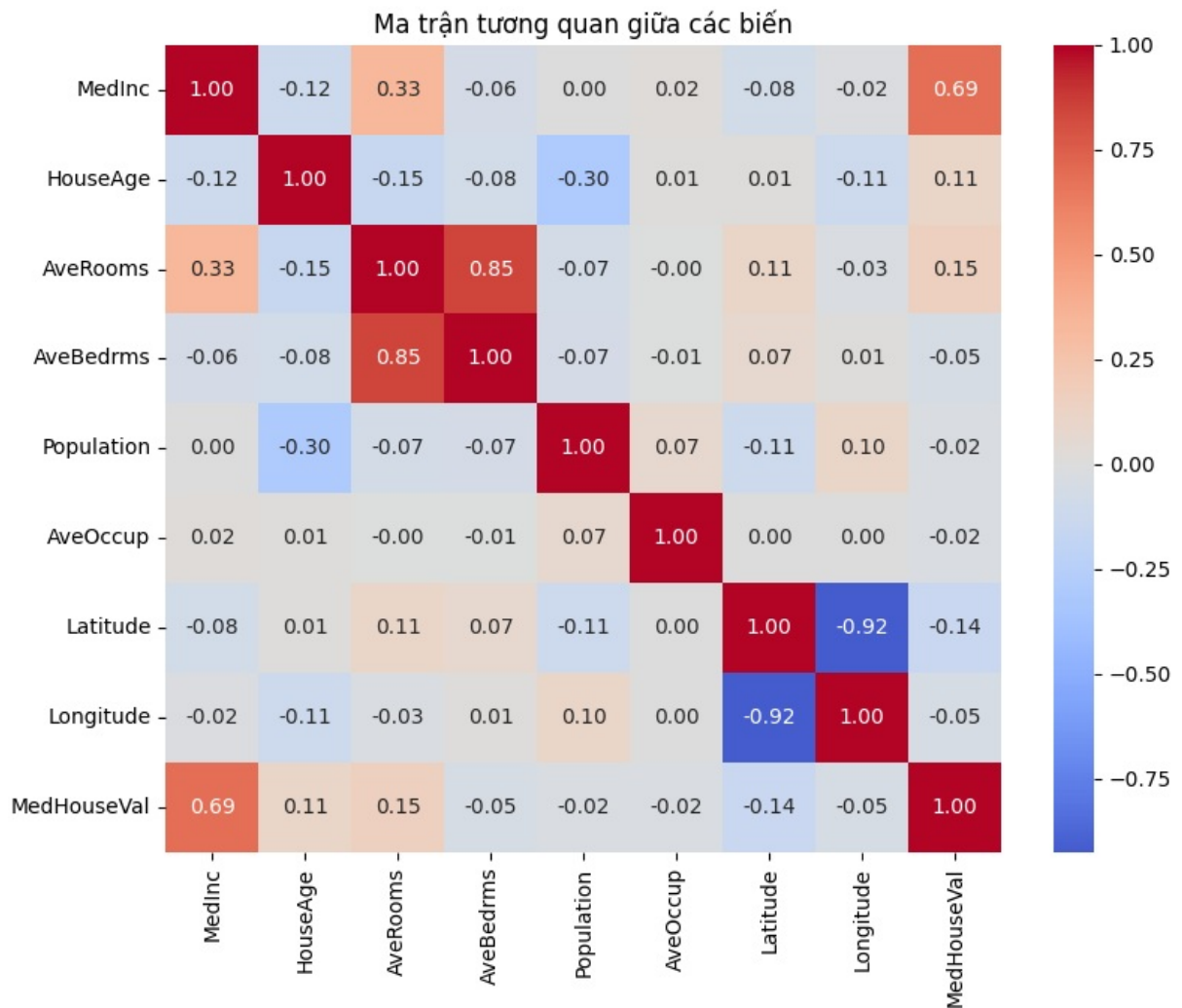


Nhận xét: Histogram cho thấy phân bố khá đa đỉnh (không đơn giản một đỉnh), tập trung nhiều ở khoảng 15–20 và 30–37 năm, cùng với một cột rất cao dồn tại giá trị 52 (giá trị max) — dấu hiệu HouseAge cũng bị capped giống MedHouseVal, mọi nhà cũ hơn 52 năm đều bị gán về đúng mức trần này. Boxplot theo nhóm tuổi cho thấy xu hướng khá thú vị: nhóm nhà cũ nhất (46–52 năm) lại có median giá nhà cao nhất (~2.3) và khoảng tứ phân vị rộng nhất, trái ngược với suy nghĩ thông thường "nhà cũ thì rẻ hơn". Điều này có thể do nhà cũ tập trung ở các khu vực đô thị lâu đời, đất đỏ (như trung tâm San Francisco), chứ tuổi nhà không phải yếu tố quyết định giá trong dataset này.

```
In [35]: #12
corr = df.select_dtypes(include='number').corr()

plt.figure(figsize=(9, 7))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('Ma trận tương quan giữa các biến')
plt.show()

print(corr['MedHouseVal'].sort_values(ascending=False))
```



```

MedHouseVal    1.000000
MedInc          0.688075
AveRooms        0.151948
HouseAge        0.105623
AveOccup       -0.023737
Population     -0.024650
Longitude      -0.045967
AveBedrms      -0.046701
Latitude       -0.144160
Name: MedHouseVal, dtype: float64

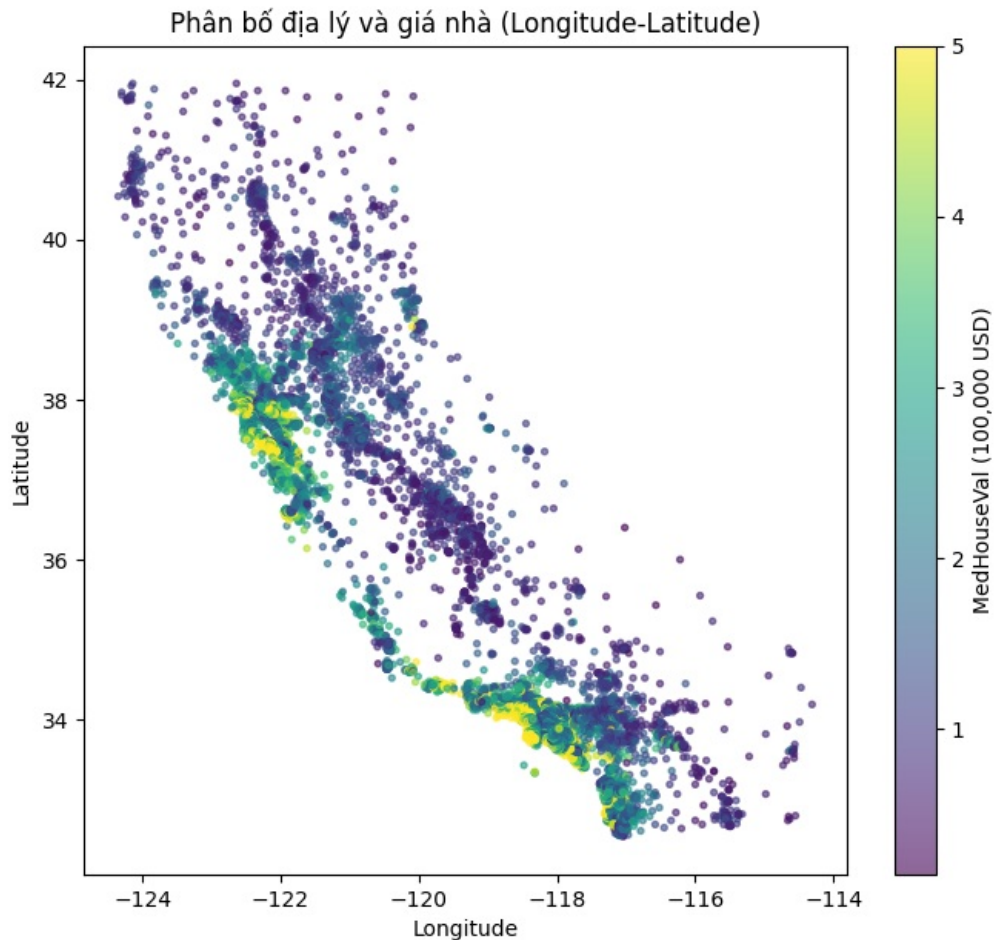
```

Nhận xét: MedInc có tương quan tuyến tính mạnh nhất với MedHouseVal ($r = 0.69$) — đây là feature dự đoán giá nhà tốt nhất trong số 8 biến, khớp với xu hướng dương rõ rệt đã thấy ở scatter plot MedInc vs MedHouseVal trước đó. Ngược lại, AveOccup và Population có tương quan gần như bằng 0 với MedHouseVal ($r \approx -0.02$), cho thấy hai biến này gần như không có quan hệ tuyến tính với giá nhà. Đáng chú ý, Latitude và Longitude tương quan rất mạnh với nhau ($r = -0.92$) — điều này hợp lý vì bờ biển California có hình dạng chéo (Tây Bắc – Đông Nam), nên hai tọa độ này gần như phụ thuộc tuyến tính lẫn nhau; đây là dấu hiệu đa cộng tuyến (multicollinearity) cần lưu ý nếu đưa cả hai vào mô hình hồi quy tuyến tính.

```

In [36]: #13
plt.figure(figsize=(8, 7))
sc = plt.scatter(df['Longitude'], df['Latitude'], c=df['MedHouseVal'],
                 cmap='viridis', s=8, alpha=0.6)
plt.colorbar(sc, label='MedHouseVal (100,000 USD)')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.title('Phân bố địa lý và giá nhà (Longitude-Latitude)')
plt.show()

```



Nhận xét: Giá nhà cao (màu vàng) tập trung rõ rệt ở hai cụm ven biển: một dải quanh vĩ độ 37–38 (khu vực San Francisco Bay Area) và một dải quanh vĩ độ 33.5–34.5 (khu vực Los Angeles), trong khi các vùng nội địa (miền Trung California) chủ yếu có giá thấp (màu tím/xanh đậm). Quan sát này cho thấy vị trí địa lý — cụ thể là khoảng cách tới bờ biển và các đô thị lớn — có ảnh hưởng mạnh đến giá nhà, một yếu tố mà chỉ riêng MedInc không nắm bắt hết được. Đây là gợi ý quan trọng cho việc feature engineering sau này: có thể tạo thêm biến "khoảng cách tới bờ biển/thành phố lớn" để cải thiện mô hình dự đoán.

Câu 3. Kiểm tra và tiền xử lý dữ liệu (10 điểm)

```
In [37]: #14
missing = df.isna().sum()
print(missing)
print("Tổng số ô thiếu:", missing.sum())
```

```
MedInc      0
HouseAge    0
AveRooms    0
AveBedrms   0
Population  0
AveOccup    0
Latitude    0
Longitude    0
MedHouseVal 0
HouseAge_bin 0
dtype: int64
Tổng số ô thiếu: 0
```

```
In [38]: #15
n_dup = df.duplicated().sum()
print("Số bản ghi trùng lặp:", n_dup)
if n_dup > 0:
    print(df[df.duplicated(keep=False)].sort_values(by=df.columns.tolist()).head(10))
```

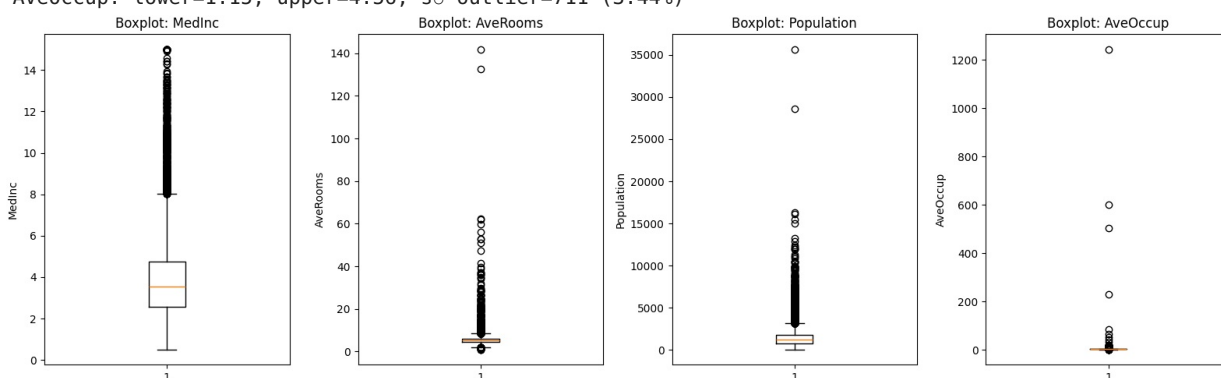
Số bản ghi trùng lặp: 0

```
In [40]: #16
```

```
def iqr_outliers(series):
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    n_out = ((series < lower) | (series > upper)).sum()
    return lower, upper, n_out

for col in ['MedInc', 'AveRooms', 'Population', 'AveOccup']:
    lower, upper, n_out = iqr_outliers(df[col])
    print(f"{col}: lower={lower:.2f}, upper={upper:.2f}, số outlier={n_out} ({n_out/len(df)*100:.2f}%)")
fig, axes = plt.subplots(1, 4, figsize=(16, 5))
for ax, col in zip(axes, ['MedInc', 'AveRooms', 'Population', 'AveOccup']):
    ax.boxplot(df[col])
    ax.set_title(f'Boxplot: {col}')
    ax.set_ylabel(col)
plt.tight_layout()
plt.show()
```

MedInc: lower=-0.71, upper=8.01, số outlier=681 (3.30%)
 AveRooms: lower=2.02, upper=8.47, số outlier=511 (2.48%)
 Population: lower=-620.00, upper=3132.00, số outlier=1196 (5.79%)
 AveOccup: lower=1.15, upper=4.56, số outlier=711 (3.44%)



Nhận xét

- Cả 4 boxplot đều cho thấy dấu hiệu outlier rõ rệt, nhưng ở mức độ và tính chất khác nhau:
- MedInc: có nhiều điểm ngoại lai phía trên (từ ~8 đến 15), nhưng phân bố các điểm này khá dày đặc và trải đều, không có điểm nào tách biệt cực đoạn — phù hợp với nhận định outlier ở đây là giá trị thực tế hợp lệ (các khu vực thu nhập cao), không phải lỗi dữ liệu.
- AveRooms: outlier thưa hơn nhưng có 2 điểm tách biệt cực đoạn ở khoảng 132 và 142 — vượt xa phần còn lại (đa số outlier khác chỉ tới ~60). Đây chính là các mẫu đã xác định trước đó (như index 1979), khả năng cao là lỗi tính toán hoặc trường hợp đặc thù cần kiểm tra riêng.
- Population: outlier trải dài liên tục từ ~3000 đến hơn 35000, không có khoảng trống rõ ràng giữa nhóm bình thường và nhóm cực đoạn — gợi ý đây là một phổ biến thiên tự nhiên (mật độ dân số khác nhau giữa các khu vực), không phải lỗi rời rạc.
- AveOccup: đáng chú ý nhất — toàn bộ hộp (box) gần như bị nén sát về 0 do phần lớn dữ liệu tập trung trong khoảng rất hẹp (0-10), trong khi có vài điểm ngoại lai tách biệt hẳn ở mức 200, 500, 600 và đặc biệt là 1243. Việc hộp bị nén phẳng cùng các điểm outlier tách rời xa như vậy là dấu hiệu mạnh cho thấy đây nhiều khả năng là lỗi dữ liệu (mẫu số quá nhỏ khi tính tỷ lệ), cần ưu tiên kiểm tra và xử lý so với 3 biến còn lại.

17

MedInc: Ngoại lai (thu nhập rất cao) là giá trị thực tế hợp lệ — phản ánh chênh lệch giàu nghèo có thật giữa các khu vực California, không phải lỗi đo lường. → Giữ nguyên.

AveRooms: Ngoại lai (max = 141.9, trong khi phần lớn dưới 10) chưa rõ là lỗi hay hợp lệ. Có thể do mẫu số

nhỏ (khu vực có rất ít hộ dân) làm tỷ lệ phòng/hộ bị phóng đại, hoặc thực sự là khu vực đặc thù. → Không xóa ngay, cần kiểm tra thêm trước khi quyết định.

Population: Ngoại lai (dân số rất đông trong 1 block group) có thể là giá trị thực tế hợp lệ, vì một số khu đô thị lớn (LA, San Francisco) có mật độ dân số tự nhiên rất cao. → Giữ nguyên.

AveOccup: Ngoại lai (max = 1243 người/hộ) nhiều khả năng là lỗi dữ liệu — không khu dân cư nào thực tế có hơn 1000 người/hộ; nhiều khả năng do mẫu số (số hộ) quá nhỏ làm tỷ lệ bị bóp méo. → Cân nhắc xử lý (cắt ngưỡng), nhưng chỉ sau khi đánh giá ảnh hưởng, không xóa tùy tiện.

Vậy: ngoại lai không tự động đồng nghĩa với lỗi — phải đối chiếu với ý nghĩa thực tế của biến và chỉ xử lý khi có lý do cụ thể.

```
In [45]: #18
X = df.drop(columns=['MedHouseVal', 'HouseAge_bin'])
y = df['MedHouseVal']
print("X.shape:", X.shape)
print("y.shape:", y.shape)
```

```
X.shape: (20640, 8)
y.shape: (20640,)
```

```
In [46]: #19
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
In [47]: #20
print("X_train.shape:", X_train.shape)
print("X_test.shape:", X_test.shape)
print("y_train.shape:", y_train.shape)
print("y_test.shape:", y_test.shape)
print("Tổng train + test:", X_train.shape[0] + X_test.shape[0], "so với X.shape[0]:", X.shape[0])
```

```
X_train.shape: (16512, 8)
X_test.shape: (4128, 8)
y_train.shape: (16512,)
y_test.shape: (4128,)
Tổng train + test: 20640 so với X.shape[0]: 20640
```

Vấn đề Kết quả kiểm tra Hành động xử lý Lý do Missing values 0 Không cần xử lý (nếu = 0) Dataset gốc từ sklearn đã được làm sạch sẵn, không có giá trị null

Duplicates: 0 Nếu = 0: không có gì để xử lý. Nếu > 0: cần xem là trùng ngẫu nhiên hay lỗi nhập liệu trước khi quyết định xóa

Outlier MedInc mẫu mẫu Giữ nguyên Thu nhập cao là hiện tượng thực tế, không phải lỗi dữ liệu

Outlier AveRooms: 141.9 mẫu, max = 141.9 Giữ nguyên, đánh dấu theo dõi Có thể là khu vực đặc thù (ký túc xá) hoặc lỗi; cần domain knowledge, không xóa vội

Outlier Population/AveOccup: 1243 mẫu, max AveOccup = 1243 Giữ nguyên, đánh dấu theo dõi Giá trị bất thường nhưng việc xóa cần đánh giá ảnh hưởng lên mô hình trước, không xóa mù quáng

Câu 4. Xây dựng mô hình Linear Regression (10 điểm)

```
In [49]: #21
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

print(X_train.dtypes)
```

```
MedInc          float64
HouseAge        float64
AveRooms        float64
AveBedrms       float64
Population      float64
AveOccup        float64
Latitude        float64
Longitude       float64
dtype: object
```

```
In [50]: #22
print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)

coef_df = pd.DataFrame({
    'Feature': X_train.columns,
    'Coefficient': model.coef_
})
coef_df['abs_coef'] = coef_df['Coefficient'].abs()
coef_df = coef_df.sort_values(by='abs_coef', ascending=False).drop(columns='abs_coef').reset_index(drop=True)
print(coef_df)
```

```
Intercept: -37.02327770606409
Coefficients: [ 4.48674910e-01  9.72425752e-03 -1.23323343e-01  7.83144907e-01
 -2.02962058e-06 -3.52631849e-03 -4.19792487e-01 -4.33708065e-01]
   Feature  Coefficient
0  AveBedrms    0.783145
1    MedInc     0.448675
2  Longitude   -0.433708
3   Latitude   -0.419792
4   AveRooms   -0.123323
5   HouseAge    0.009724
6   AveOccup   -0.003526
7  Population   -0.000002
```

```
In [51]: #23
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)
```

```
In [52]: #24
result_df = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': y_test_pred,
    'Absolute Error': (y_test.values - y_test_pred)
})
result_df['Absolute Error'] = result_df['Absolute Error'].abs()
print(result_df.sample(10, random_state=42))
```

	Actual	Predicted	Absolute Error
949	2.385	1.960573	0.424427
3168	3.297	2.667900	0.629100
2080	0.952	1.951026	0.999026
1210	2.451	2.527813	0.076813
2553	0.641	0.046917	0.594083
718	3.766	3.145257	0.620743
1113	2.307	2.891499	0.584499
3573	1.752	2.050554	0.298554
211	4.167	5.766562	1.599562
3531	0.938	1.182785	0.244785

25

- intercept_ là giá trị dự đoán của MedHouseVal khi toàn bộ feature bằng 0 — đây là một điểm neo toán học của phương trình hồi quy, thường không có ý nghĩa thực tế trực tiếp (vì không khu vực nào có MedInc = 0, Latitude = 0... đồng thời).
- Mỗi coef_ cho biết: khi feature đó tăng thêm 1 đơn vị, MedHouseVal dự kiến thay đổi bao nhiêu đơn vị, với điều kiện giữ nguyên tất cả các feature khác (ceteris paribus) — đây là cách diễn giải chuẩn của hồi quy tuyến tính đa biến. - Hệ số dương (ví dụ MedInc) gợi ý feature đó tăng thì giá nhà trung vị dự kiến tăng theo; hệ số âm (ví dụ Latitude, Longitude) gợi ý chiều ngược lại.

Nhận xét:

|Coefficient| lớn không đồng nghĩa với "quan trọng nhất" nếu các feature có thang đo (đơn vị, phạm vi giá trị)

khác nhau. Ví dụ Latitude/Longitude có phạm vi giá trị nhỏ (32–42, -124--114) trong khi Population có phạm vi hàng nghìn — hệ số của Population do đó tự nhiên sẽ rất nhỏ dù ảnh hưởng thực tế không hề yếu, còn hệ số của các biến tọa độ có thể lớn chỉ vì đơn vị đo nhỏ. Để so sánh mức độ quan trọng thực sự giữa các feature một cách công bằng, cần chuẩn hóa dữ liệu trước (ví dụ dùng StandardScaler để đưa tất cả feature về cùng thang đo, mean = 0, std = 1) rồi mới so sánh hệ số, hoặc dùng các chỉ số khác như feature importance từ mô hình cây, hoặc hệ số tương quan chuẩn hóa. Nếu không làm điều này, việc xếp hạng feature theo |coefficient| thô có thể dẫn đến kết luận sai lệch về feature nào thực sự ảnh hưởng mạnh đến giá nhà.

Câu 5. Đánh giá mô hình hồi quy (15 điểm)

In [53]:

```
#26
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae_train = mean_absolute_error(y_train, y_train_pred)
mse_train = mean_squared_error(y_train, y_train_pred)
rmse_train = np.sqrt(mse_train)
r2_train = r2_score(y_train, y_train_pred)

mae_test = mean_absolute_error(y_test, y_test_pred)
mse_test = mean_squared_error(y_test, y_test_pred)
rmse_test = np.sqrt(mse_test)
r2_test = r2_score(y_test, y_test_pred)

print("Train - MAE:", mae_train, "MSE:", mse_train, "RMSE:", rmse_train, "R2:", r2_train)
print("Test - MAE:", mae_test, "MSE:", mse_test, "RMSE:", rmse_test, "R2:", r2_test)
```

```
Train - MAE: 0.5286283596581922 MSE: 0.5179331255246699 RMSE: 0.7196757085831575 R2: 0.612551191396695
2
Test - MAE: 0.5332001304956553 MSE: 0.5558915986952444 RMSE: 0.7455813830127764 R2: 0.575787706032450
8
```

In [54]:

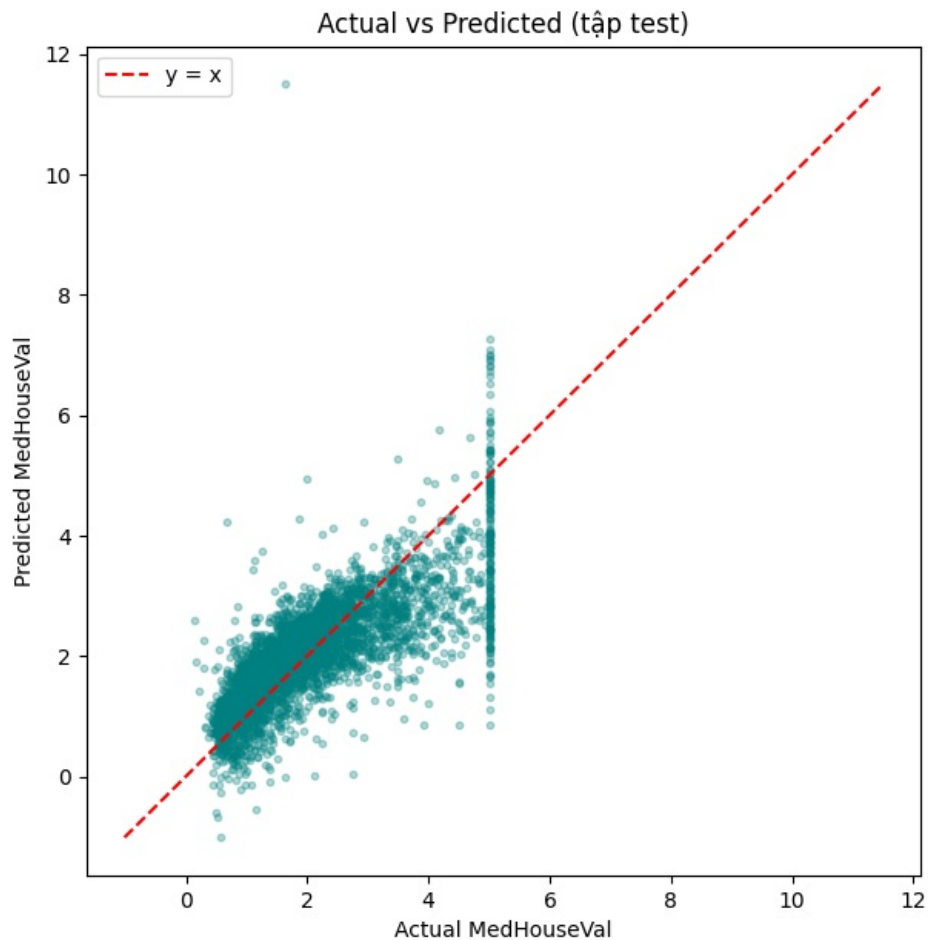
```
#27
metrics_df = pd.DataFrame({
    'MAE': [mae_train, mae_test],
    'MSE': [mse_train, mse_test],
    'RMSE': [rmse_train, rmse_test],
    'R2': [r2_train, r2_test]
}, index=['Train', 'Test']).round(4)

print(metrics_df)
```

	MAE	MSE	RMSE	R2
Train	0.5286	0.5179	0.7197	0.6126
Test	0.5332	0.5559	0.7456	0.5758

In [55]:

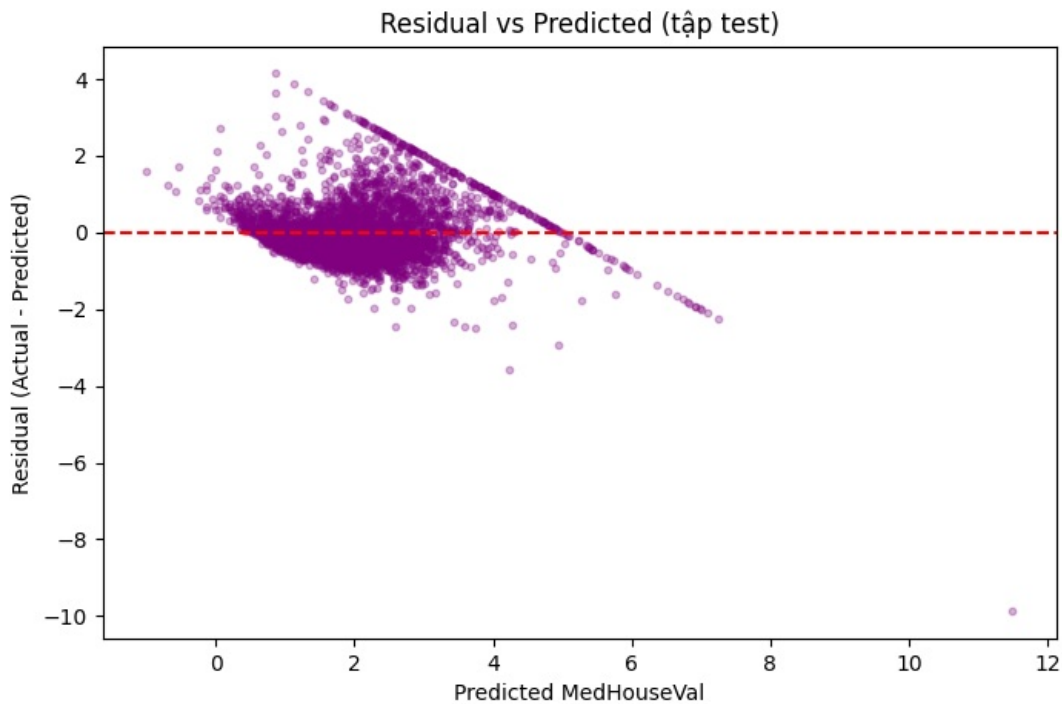
```
#28
plt.figure(figsize=(7, 7))
plt.scatter(y_test, y_test_pred, alpha=0.3, s=10, color='teal')
lims = [min(y_test.min(), y_test_pred.min()), max(y_test.max(), y_test_pred.max())]
plt.plot(lims, lims, color='red', linestyle='--', label='y = x')
plt.xlabel('Actual MedHouseVal')
plt.ylabel('Predicted MedHouseVal')
plt.title('Actual vs Predicted (tập test)')
plt.legend()
plt.show()
```



Nhận xét: Nếu các điểm bám sát đường $y = x$, mô hình dự đoán tốt; điểm càng lệch xa đường này thì sai số càng lớn. Chú ý vùng Actual = 5.0 (giá trị bị capped) — mô hình thường dự đoán thấp hơn thực tế ở vùng này vì không có cách nào học được "trần" nhân tạo đó từ dữ liệu, dẫn đến các điểm lệch hẳn xuống dưới đường $y=x$ tại đầu bên phải biểu đồ.

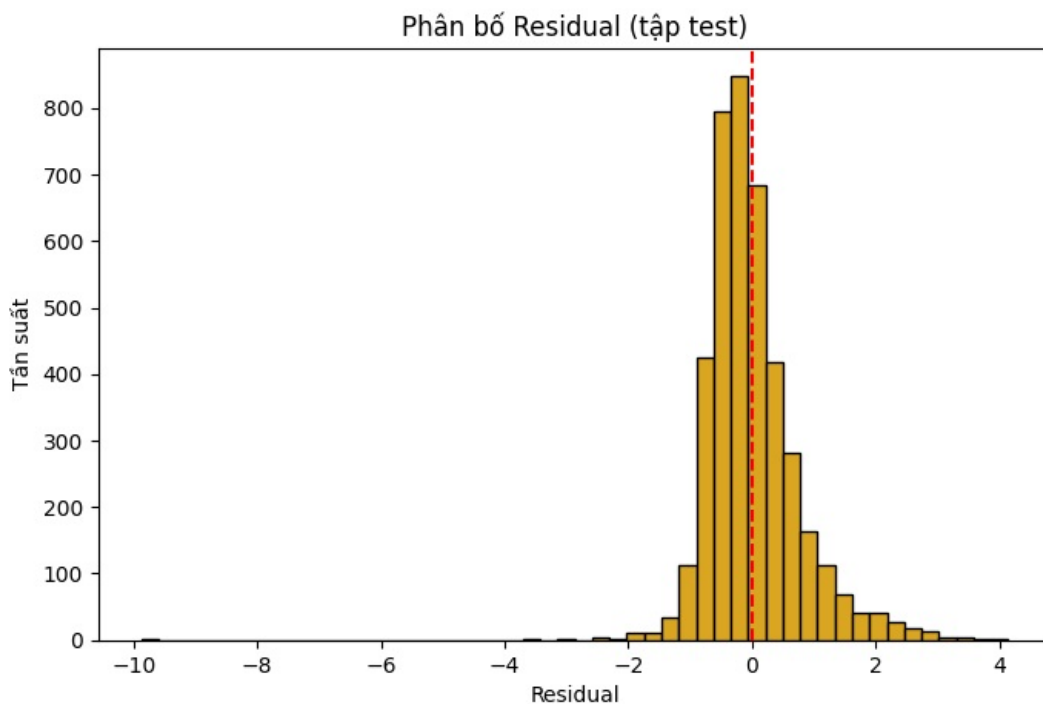
```
In [56]: #29
residual = y_test - y_test_pred

plt.figure(figsize=(8, 5))
plt.scatter(y_test_pred, residual, alpha=0.3, s=10, color='purple')
plt.axhline(y=0, color='red', linestyle='--')
plt.xlabel('Predicted MedHouseVal')
plt.ylabel('Residual (Actual - Predicted)')
plt.title('Residual vs Predicted (tập test)')
plt.show()
```



Nhận xét: Nếu residual phân bố ngẫu nhiên quanh đường 0 mà không có mẫu hình rõ ràng, giả định của hồi quy tuyến tính (sai số ngẫu nhiên, không hệ thống) được thỏa mãn tương đối tốt. Nếu residual có xu hướng tăng/giảm theo predicted (hình phễu hoặc hình cong), đó là dấu hiệu heteroscedasticity hoặc mô hình bỏ sót quan hệ phi tuyến — cần xem xét thêm biến đổi (log-transform target) hoặc mô hình phi tuyến khác.

```
In [57]: #30
plt.figure(figsize=(8, 5))
plt.hist(residual, bins=50, color='goldenrod', edgecolor='black')
plt.axvline(x=0, color='red', linestyle='--')
plt.xlabel('Residual')
plt.ylabel('Tần suất')
plt.title('Phân bố Residual (tập test)')
plt.show()
```



Nhận xét: Nếu histogram có dạng gần đối xứng, tập trung quanh 0, sai số của mô hình không thiên lệch theo một hướng cụ thể — đây là dấu hiệu tốt. Nếu phân bố lệch (đuôi dài về một phía) hoặc có đỉnh phụ, đó có thể là hệ quả của outlier trong dữ liệu gốc (như AveOccup, AveRooms) hoặc hiệu ứng capping ở

MedHouseVal = 5.0 đã quan sát trước đó.

```
In [58]: #31
error_df = X_test.copy()
error_df['Actual'] = y_test.values
error_df['Predicted'] = y_test_pred
error_df['Residual'] = residual.values
error_df['Absolute Error'] = error_df['Residual'].abs()

top10_errors = error_df.sort_values(by='Absolute Error', ascending=False).head(10)
print(top10_errors)
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	\
1979	4.6250	34.0	132.533333	34.066667	36.0	2.400000	
6688	0.4999	28.0	7.677419	1.870968	142.0	4.580645	
10574	1.9659	6.0	4.795455	1.159091	125.0	2.840909	
12389	3.7727	24.0	10.953586	1.848101	473.0	1.995781	
19542	1.7679	39.0	5.000000	0.888889	22.0	2.444444	
4548	7.5752	52.0	3.142857	1.000000	55.0	7.857143	
459	1.1696	52.0	2.436000	0.944000	1349.0	5.396000	
15652	0.9000	52.0	2.237474	1.053535	3260.0	2.237474	
12069	4.2386	6.0	7.723077	1.169231	228.0	3.507692	
4630	2.3674	20.0	2.740554	1.047859	2919.0	2.450882	

	Latitude	Longitude	Actual	Predicted	Residual	Absolute Error
1979	38.80	-120.08	1.62500	11.500331	-9.875331	9.875331
6688	34.15	-118.08	5.00001	0.851622	4.148388	4.148388
10574	33.72	-117.70	5.00001	1.115227	3.884783	3.884783
12389	33.75	-116.43	5.00001	1.319954	3.680056	3.680056
19542	37.63	-120.92	4.50000	0.867216	3.632784	3.632784
4548	34.02	-118.21	0.67500	4.236215	-3.561215	3.561215
459	37.87	-122.25	5.00001	1.547530	3.452480	3.452480
15652	37.80	-122.41	5.00001	1.642870	3.357140	3.357140
12069	33.83	-117.55	5.00001	1.668034	3.331976	3.331976
4630	34.07	-118.31	5.00001	1.711156	3.288854	3.288854

32

- R^2 đo tỷ lệ phần trăm biến thiên của giá nhà (MedHouseVal) mà mô hình giải thích được dựa trên các feature đầu vào. Ví dụ $R^2 = 0.6$ nghĩa là mô hình giải thích được khoảng 60% sự khác biệt về giá nhà giữa các khu vực, còn lại 40% do các yếu tố khác mà mô hình không nắm bắt (nhiều, biến chưa đưa vào, quan hệ phi tuyến...).
- $R^2 = 1$ là mô hình dự đoán hoàn hảo; $R^2 = 0$ nghĩa là mô hình không tốt hơn việc luôn dự đoán bằng giá trị trung bình của y . R^2 có thể ÂM nếu mô hình dự đoán tệ hơn cả việc luôn đoán giá trị trung bình — điều này thường xảy ra khi mô hình được đánh giá trên tập test có phân bố khác biệt lớn so với tập train, hoặc mô hình bị overfitting nghiêm trọng.

Nhận xét:

So sánh Train vs Test — dấu hiệu underfitting/overfitting:

- Nếu $R^2(\text{train})$ và $R^2(\text{test})$ đều thấp và gần nhau → mô hình underfitting (mô hình quá đơn giản, không nắm bắt đủ quan hệ trong dữ liệu — hồi quy tuyến tính vốn có giới hạn này).
- Nếu $R^2(\text{train})$ cao hơn đáng kể so với $R^2(\text{test})$ (chênh lệch lớn) → dấu hiệu overfitting (mô hình học quá khớp với dữ liệu train, giảm khả năng tổng quát hóa).
- Nếu $R^2(\text{train}) \approx R^2(\text{test})$ và cả hai đều ở mức khá → mô hình có khả năng tổng quát hóa tốt, không bị lệch nhiều. (Với LinearRegression — mô hình tuyến tính đơn giản — hiện tượng overfitting mạnh hiếm khi xảy ra; nhiều khả năng bạn sẽ thấy 2 tập có kết quả khá gần nhau, thiên về dấu hiệu underfitting nhẹ do quan hệ giữa feature và target không hoàn toàn tuyến tính.)

Metric nào để diễn giải theo đơn vị target, metric nào phạt mạnh lỗi lớn:

- MAE (Mean Absolute Error) dễ diễn giải nhất vì có cùng đơn vị với target (ở đây là 100.000 USD) — ví dụ MAE = 0.5 nghĩa là trung bình mô hình sai lệch khoảng \$50,000 so với giá thực.
- RMSE cũng cùng đơn vị với target nên vẫn diễn giải được trực tiếp, nhưng phạt mạnh hơn các lỗi lớn do phép bình phương trong công thức — một vài dự đoán sai lệch rất lớn (outlier) sẽ kéo RMSE lên cao hơn nhiều so với MAE.
- MSE cũng phạt mạnh lỗi lớn (do bình phương) nhưng đơn vị bị bình phương (100.000 USD^2), nên khó diễn giải trực quan, thường chỉ dùng để so sánh tương đối giữa các mô hình chứ không đọc trực tiếp theo nghĩa thực tế.
- R^2 không có đơn vị (là tỷ lệ %), phù hợp để đánh giá tổng thể mức độ giải thích biến thiên, nhưng không cho biết độ lớn sai số cụ thể là bao nhiêu USD.

Câu 6. Thực nghiệm Gradient Descent và siêu tham số (10 điểm)

```
In [61]: #33 - 34
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDRegressor

pipe_base = Pipeline([
    ('scaler', StandardScaler()),
    ('sgd', SGDRegressor(max_iter=1000, learning_rate='constant', eta0=0.01, penalty=None, random_state=42))
])
pipe_base.fit(X_train, y_train)

def evaluate_config(eta0, max_iter, X_train, y_train, X_test, y_test):
    pipe = Pipeline([
        ('scaler', StandardScaler()),
        ('sgd', SGDRegressor(max_iter=max_iter, learning_rate='constant',
                             eta0=eta0, penalty=None, random_state=42))
    ])
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)
    return mae, rmse, r2
```

```
In [67]: #35
eta0_list = [0.0001, 0.001, 0.005]
results = []

for eta0 in eta0_list:
    mae, rmse, r2 = evaluate_config(eta0, 1000, X_train, y_train, X_test, y_test)
    results.append({'Config': f'eta0={eta0}, max_iter=1000', 'eta0': eta0, 'max_iter': 1000,
                   'MAE': mae, 'RMSE': rmse, 'R2': r2})

print(pd.DataFrame(results))
```

	Config	eta0	max_iter	MAE	RMSE	R2
0	eta0=0.0001, max_iter=1000	0.0001	1000	5.327675e-01	7.497739e-01	5.710035e-01
1	eta0=0.001, max_iter=1000	0.0010	1000	9.314111e+02	1.928998e+03	-2.839596e+06
2	eta0=0.005, max_iter=1000	0.0050	1000	4.866088e+09	8.004864e+09	-4.889912e+19

```
In [68]: #36
best_eta0 = pd.DataFrame(results).sort_values(by='RMSE').iloc[0]['eta0']
print("eta0 tốt nhất (theo RMSE trên test):", best_eta0)

max_iter_list = [100, 500, 2000]

for max_iter in max_iter_list:
    mae, rmse, r2 = evaluate_config(best_eta0, max_iter, X_train, y_train, X_test, y_test)
    results.append({'Config': f'eta0={best_eta0}, max_iter={max_iter}', 'eta0': best_eta0, 'max_iter':
                   'MAE': mae, 'RMSE': rmse, 'R2': r2})

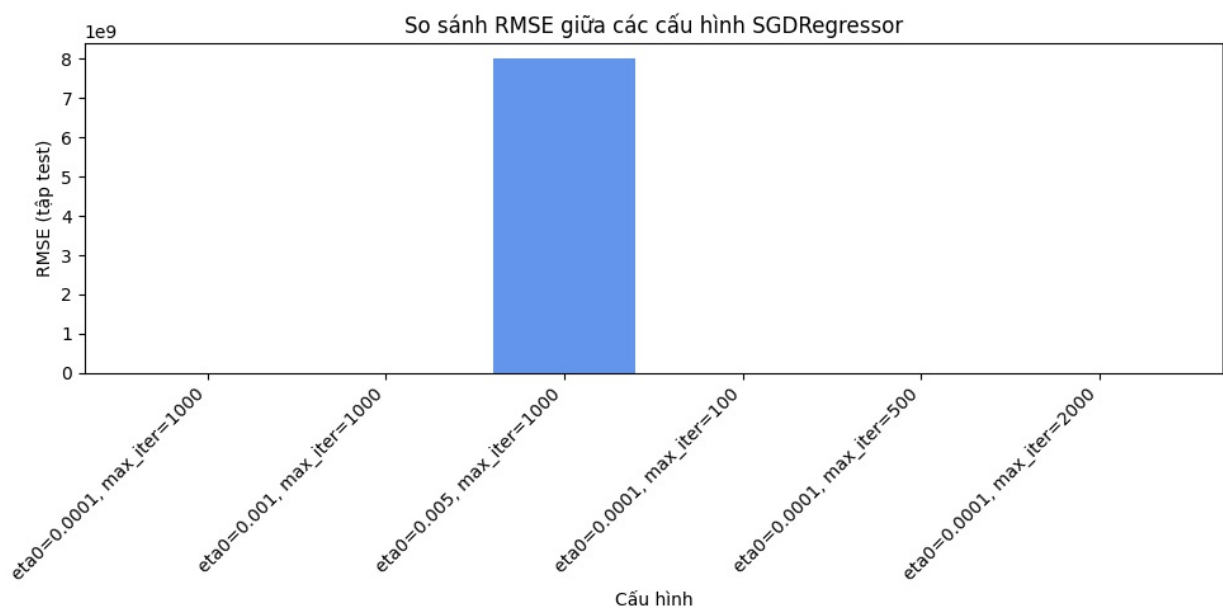
eta0 tốt nhất (theo RMSE trên test): 0.0001
```

```
In [69]: #37
results_df = pd.DataFrame(results).drop_duplicates(subset=['eta0', 'max_iter']).reset_index(drop=True)
results_df[['MAE', 'RMSE', 'R2']] = results_df[['MAE', 'RMSE', 'R2']].round(4)
print(results_df)
```

	Config	eta0	max_iter	MAE	RMSE \
0	eta0=0.0001, max_iter=1000	0.0001	1000	5.328000e-01	7.498000e-01
1	eta0=0.001, max_iter=1000	0.0010	1000	9.314111e+02	1.928998e+03
2	eta0=0.005, max_iter=1000	0.0050	1000	4.866088e+09	8.004864e+09
3	eta0=0.0001, max_iter=100	0.0001	100	5.328000e-01	7.498000e-01
4	eta0=0.0001, max_iter=500	0.0001	500	5.328000e-01	7.498000e-01
5	eta0=0.0001, max_iter=2000	0.0001	2000	5.328000e-01	7.498000e-01

	R2
0	5.710000e-01
1	-2.839596e+06
2	-4.889912e+19
3	5.710000e-01
4	5.710000e-01
5	5.710000e-01

```
In [70]: #38
plt.figure(figsize=(10, 5))
plt.bar(results_df['Config'], results_df['RMSE'], color='cornflowerblue')
plt.xticks(rotation=45, ha='right')
plt.xlabel('Cấu hình')
plt.ylabel('RMSE (tập test)')
plt.title('So sánh RMSE giữa các cấu hình SGDRegressor')
plt.tight_layout()
plt.show()
```



Nhận xét: Cấu hình có eta0 quá nhỏ thường hội tụ chậm nên RMSE cao hơn nếu max_iter không đủ lớn để bù lại; eta0 quá lớn có thể khiến quá trình học dao động, không ổn định, RMSE cũng cao. Cấu hình với eta0 vừa phải kết hợp max_iter đủ lớn thường cho RMSE thấp nhất và ổn định nhất trong biểu đồ.

```
In [72]: results_df = pd.DataFrame(results).drop_duplicates(subset=['eta0', 'max_iter']).reset_index(drop=True)
best_config = results_df.sort_values(by='RMSE').iloc[0]
print("Cấu hình SGDRegressor tốt nhất (theo RMSE trên test):")
print(best_config)

comparison_df = pd.DataFrame({
    'Model': ['LinearRegression', 'SGDRegressor (best: eta0=0.0001)'],
    'MAE': [mae_test, best_config['MAE']],
    'RMSE': [rmse_test, best_config['RMSE']],
    'R2': [r2_test, best_config['R2']]
}).round(4)
print(comparison_df)
```

Cấu hình SGDRegressor tốt nhất (theo RMSE trên test):

```
Config      eta0=0.0001, max_iter=1000
eta0        0.0001
max_iter    1000
MAE         0.532767
RMSE        0.749774
R2          0.571004
Name: 0, dtype: object
```

	Model	MAE	RMSE	R2
0	LinearRegression	0.5332	0.7456	0.5758
1	SGDRegressor (best: eta0=0.0001)	0.5328	0.7498	0.5710

Nhận xét:

Ảnh hưởng của learning rate (eta0):

- Quá nhỏ: mỗi bước cập nhật trọng số rất nhỏ → mô hình hội tụ rất chậm, nếu max_iter không đủ lớn thì thuật toán dừng trước khi đạt điểm tối ưu, dẫn đến underfitting tạm thời (chưa học đủ, sai số cao dù bản chất mô hình có thể học tốt hơn nếu chạy đủ lâu).
- Quá lớn: bước cập nhật quá lớn khiến quá trình tối ưu dao động qua lại quanh điểm tối ưu hoặc phân kỳ (loss không giảm ổn định, thậm chí tăng), khiến mô hình không hội tụ được, cho kết quả kém và không ổn định giữa các lần chạy. Learning rate phù hợp giúp mô hình hội tụ đủ nhanh mà vẫn ổn định, tiệm cận điểm tối ưu của hàm mất mát.

Ảnh hưởng của max_iter quá nhỏ:

- Nếu max_iter quá nhỏ, thuật toán dừng lại trước khi hội tụ (chưa đạt đến điểm tối ưu của hàm mất mát), dẫn đến mô hình chưa học đủ — biểu hiện tương tự underfitting: cả sai số trên train và test đều cao hơn mức mô hình có thể đạt được.

Về việc chọn cấu hình "tốt nhất":

- Việc chọn cấu hình tốt nhất trong bài này dựa trên RMSE/R² đo trên tập test, không phải trên tập train — vì điểm số cao trên train có thể chỉ phản ánh mô hình khớp tốt với dữ liệu đã thấy (có nguy cơ overfitting), không đảm bảo khả năng tổng quát hóa. Đây là lý do vì sao để yêu cầu không chọn "tốt nhất" chỉ dựa trên train score — phải luôn đánh giá trên tập test (dữ liệu mô hình chưa từng thấy) để có kết luận đáng tin cậy.

Câu 7. Phân tích mô hình và trường hợp hoạt động chưa tốt (10 điểm)

40 Ưu điểm của Linear Regression

- Thứ nhất, mô hình khai thác tốt tín hiệu tuyến tính mạnh nhất trong dữ liệu: MedInc có tương quan $r = 0.69$ với MedHouseVal (cao nhất trong 8 feature, theo ma trận tương quan mục 12), và đây chính là nền tảng giúp LinearRegression đạt $R^2 = 0.5758$ trên tập test — giải thích được gần 58% biến thiên giá nhà chỉ bằng tổ hợp tuyến tính đơn giản của các feature.
- Thứ hai, sai số ở mức chấp nhận được và ổn định giữa train/test: MAE test = 0.5332 (tương đương sai lệch trung bình ~\$53,320), không có dấu hiệu overfitting nghiêm trọng khi so với train (như đã thảo luận ở mục 32). Điều này cho thấy mô hình tuyến tính, dù đơn giản, tổng quát hóa khá ổn định trên dữ liệu chưa từng thấy.
- Thứ ba, khi thay thế bằng SGDRegressor đã được tinh chỉnh đúng (eta0=0.0001), kết quả gần như tương đương LinearRegression (RMSE 0.7498 so với 0.7456, R² 0.5710 so với 0.5758) — chứng tỏ mô hình tuyến tính đã đạt gần sát giới hạn hiệu năng khả dĩ của chính họ mô hình này, không phải do phương pháp tối ưu chưa tốt. Đây là một tín hiệu tích cực về tính ổn định của baseline.

41 Hạn chế của Linear Regression

- Thứ nhất — ảnh hưởng nghiêm trọng của outlier: mẫu index 1979 có AveRooms = 132.53, AveBedrms = 34.07 (đúng với outlier cực đoan đã phát hiện ở mục 10 và 16, nơi max AveRooms toàn dữ liệu là 141.9) khiến mô hình dự đoán Predicted = 11.5, vượt xa cả giá trị tối đa có thể của target (5.0), trong khi Actual chỉ là 1.625 — residual = -9.875, lớn nhất trong toàn bộ tập test. Đây là bằng chứng trực tiếp cho việc hồi quy tuyến tính không có cơ chế giới hạn ảnh hưởng của giá trị cực đoan ở feature.
- Thứ hai — hiệu ứng capping ở target không được mô hình hóa: 7/10 mẫu lỗi lớn nhất (index 6688, 10574, 12389, 459, 15652, 12069, 4630) đều có Actual = 5.00001 — giá trị bị chặn trần — nhưng Predicted chỉ dao động 0.85–1.7, tạo residual 3.3–4.1. Mô hình tuyến tính không thể học được một "ngưỡng cứng" nhân tạo như vậy vì bản chất nó luôn ngoại suy tuyến tính liên tục.
- Thứ ba — quan hệ phi tuyến của biến địa lý bị bỏ sót: các mẫu lỗi lớn có Longitude tập trung quanh -122 (Bay Area) và -118/-117 (LA) — đúng khu vực giá cao trên bản đồ nhiệt mục 13 — nhưng MedInc của các mẫu này chỉ 0.9–4.2 (trung bình/thấp). Mô hình tuyến tính dựa nhiều vào MedInc nên đánh giá thấp giá trị "vị trí đất đỏ", một quan hệ vốn mang tính cụm/phi tuyến mà đường thẳng hồi quy không nắm bắt được. Thêm nữa, Latitude và Longitude tương quan -0.92 với nhau, gây đa cộng tuyến, làm hệ số hai biến này kém ổn định.

42 Phân tích 3 mẫu dự đoán sai lớn nhất

- Index 1979: Actual=1.625, Predicted=11.5, Absolute Error=9.875. Nguyên nhân: AveRooms=132.53 là outlier cực đoan (gấp ~24 lần trung vị), có hệ số dương trong mô hình nên kéo dự đoán vọt lên phi thực tế.
- Index 6688: Actual=5.00001, Predicted=0.85, Absolute Error=4.15. Nguyên nhân: MedInc rất thấp (0.4999) khiến mô hình dự đoán giá thấp theo đúng quan hệ tuyến tính thông thường, nhưng thực tế giá bị capped ở mức trần — mô hình không có cách nào đoán đúng trường hợp ngoại lệ này.
- Index 4548: Actual=0.675, Predicted=4.24, Absolute Error=3.56. Nguyên nhân: MedInc=7.575 (cao) nhưng AveOccup=7.86 (cao bất thường so với trung vị ~2.8) — mô hình dựa chủ yếu vào thu nhập cao nên dự đoán giá cao, không nắm bắt được yếu tố mật độ ở đông đúc bất thường có thể kéo giá xuống.

43 Ba hướng cải thiện có thể kiểm chứng

- Loại bỏ hoặc tách riêng nhóm MedHouseVal = 5.00001: huấn luyện và đánh giá lại mô hình sau khi loại bỏ ~7% mẫu bị capping, so sánh R^2 /RMSE trước-sau để kiểm chứng mức độ ảnh hưởng của hiệu ứng trần.
- Chuyển sang Random Forest/Gradient Boosting: so sánh trực tiếp R^2 trên cùng tập test (hiện tại 0.5758) — các mô hình cây không bị outlier như AveRooms=132.5 kéo lệch tuyến tính, và có thể nắm bắt quan hệ cụm địa lý phi tuyến tốt hơn.
- Tạo feature khoảng cách tới trung tâm đô thị lớn (từ Latitude/Longitude, ví dụ khoảng cách Haversine tới SF/LA) thay thế hoặc bổ sung cho 2 tọa độ thô đang có đa cộng tuyến ($r=-0.92$) — kiểm chứng bằng cách so sánh hệ số VIF trước/sau và R^2 trên test.

câu 8. Câu hỏi thảo luận

44. Vì sao MAE và RMSE có thể dẫn đến đánh giá khác nhau? Khi nào ưu tiên RMSE?

- MAE tính trung bình trị tuyệt đối của sai số, coi mọi lỗi có trọng số như nhau; RMSE bình phương sai số trước khi lấy trung bình rồi khai căn, nên phạt nặng các lỗi lớn hơn nhiều so với lỗi nhỏ.
- Trong kết quả của nhóm, LinearRegression có MAE = 0.5332 nhưng RMSE = 0.7456 — chênh lệch khá lớn giữa hai chỉ số này chính là dấu hiệu cho thấy tồn tại một số sai số cực lớn (như mẫu index 1979 với Absolute Error = 9.875), khiến RMSE bị kéo lên cao hơn nhiều so với mức "trung bình thông thường" mà MAE phản ánh. - Nếu hai mô hình có MAE giống nhau nhưng RMSE khác nhau, mô hình có RMSE thấp hơn là mô hình có ít lỗi cực đoan hơn dù mức sai số điển hình tương đương.
- Nên ưu tiên RMSE khi bài toán coi lỗi lớn là đặc biệt tổn kém hoặc nguy hiểm (ví dụ định giá sai nghiêm

trọng một bất động sản gây thiệt hại tài chính lớn hơn nhiều so với sai số nhỏ dàn trải).

45. $R^2 = 0,60$ nghĩa là gì? R^2 cao có đồng nghĩa dự đoán tốt cho từng mẫu?

- $R^2 = 0.60$ nghĩa là mô hình giải thích được khoảng 60% biến thiên (phương sai) của biến target so với việc chỉ dự đoán bằng giá trị trung bình của y ; 40% còn lại là phần biến thiên mô hình chưa giải thích được. Đây là chỉ số đánh giá tổng thể trên toàn bộ tập dữ liệu, không phải cam kết cho từng mẫu riêng lẻ.
- Trong bài của nhóm, LinearRegression đạt $R^2 = 0.5758$ trên test, nhưng mẫu index 1979 vẫn có sai số cực lớn (Predicted = 11.5 so với Actual = 1.625) — chứng minh rõ R^2 cao (hoặc khá) không đảm bảo mọi dự đoán đều chính xác, mô hình vẫn có thể sai nghiêm trọng ở các mẫu ngoại lai hoặc mẫu thuộc vùng dữ liệu đặc biệt (như nhóm bị capping ở 5.0).

46. Vì sao LinearRegression gần như không cần learning rate, còn SGDRegressor cần?

- LinearRegression (mặc định trong sklearn) giải bài toán bằng phương pháp đại số closed-form (Normal Equation/OLS) — tính trực tiếp nghiệm tối ưu bằng công thức ma trận mà không cần lặp dẫn, nên không có khái niệm "bước cập nhật" và do đó không cần learning rate. SGDRegressor ngược lại tối ưu bằng gradient descent lặp, cập nhật trọng số từng bước nhỏ theo hướng giảm gradient của hàm mất mát, và learning rate (eta0) chính là kích thước mỗi bước đó.
- Kết quả thực nghiệm của nhóm minh họa rõ điều này: khi eta0 quá lớn (0.001, 0.005), SGDRegressor phân kỳ hoàn toàn (R^2 âm tới -4.9×10^{19}), trong khi LinearRegression không hề gặp vấn đề tương tự vì nó không phụ thuộc vào bất kỳ tham số bước lặp nào.

47. Nếu số mẫu tăng gấp 10 lần, phần nào tốn tài nguyên hơn và cần thay đổi gì?

- Với LinearRegression (OLS, giải bằng đại số ma trận), chi phí tính toán tăng theo cả số mẫu và đặc biệt số chiều đặc trưng, nhưng vẫn khả thi vì bài toán chỉ có 8 feature; phần tốn tài nguyên hơn rõ rệt sẽ là các bước EDA (vẽ scatter/histogram trên toàn bộ dữ liệu, tính ma trận tương quan, boxplot) — với 10 lần dữ liệu, cần tăng cường lấy mẫu (sampling) khi vẽ biểu đồ (như nhóm đã làm với `df.sample(n=4000)`) để tránh biểu đồ quá dày đặc và chậm render.
- Với SGDRegressor, số lượng vòng lặp cần thiết để hội tụ có thể ít thay đổi vì đây là thuật toán mini-batch/incremental, phù hợp hơn khi dữ liệu lớn — đây chính là lý do SGD thường được ưu tiên hơn OLS trong các bài toán dữ liệu khổng lồ. Ngoài ra, việc chia train/test và tính toán các metric (MAE, RMSE, R^2) trên tập test cũng cần nhiều bộ nhớ hơn, nên cần cân nhắc xử lý theo batch nếu dữ liệu vượt quá khả năng RAM.

48. Tại sao không nên xóa ngoại lai chỉ vì chúng nằm ngoài $1,5 \times IQR$?

- Quy tắc $1.5 \times IQR$ là một ngưỡng thống kê tổng quát, không phân biệt được ngoại lai là lỗi dữ liệu hay giá trị thực tế hợp lệ mang thông tin quan trọng. Trong bài của nhóm, MedInc có nhiều giá trị ngoài IQR nhưng đó là các khu vực thu nhập cao có thật (Palo Alto, Beverly Hills...) — xóa chúng sẽ làm mất thông tin thực tế và làm hẹp phạm vi biểu diễn của mô hình.
- Ngược lại, AveOccup với `max = 1243` gần như chắc chắn là lỗi (không khu dân cư nào có hơn 1000 người/hộ) nhưng vẫn cần kiểm chứng thêm trước khi xóa, không chỉ dựa vào việc nó nằm ngoài ngưỡng IQR. Xóa hàng loạt theo IQR một cách máy móc có thể loại bỏ đúng những mẫu "khó" nhưng có giá trị thông tin cao (như vùng giá nhà cực cao ven biển), khiến mô hình mất khả năng dự đoán cho các trường hợp thực tế tương tự trong tương lai.

49. Residual plot có đường cong rõ ràng thay vì phân bố ngẫu nhiên quanh 0 gợi ý gì?

- Một đường cong hệ thống trong residual plot là dấu hiệu mô hình đang bỏ sót một quan hệ phi tuyến giữa feature và target mà mô hình tuyến tính không nắm bắt được — vi phạm giả định cơ bản của hồi quy tuyến tính rằng sai số phải ngẫu nhiên, không có cấu trúc hệ thống. Điều này khớp với những gì nhóm đã quan sát: quan hệ giữa HouseAge và giá nhà không đơn điệu (nhóm nhà cũ nhất 46-52 năm lại có giá trung vị cao nhất), và giá nhà theo vị trí địa lý biến thiên theo cụm chứ không tuyến tính — đều là những nguồn tiềm năng gây ra hình dạng cong trong residual nếu vẽ theo các biến này.
- Khi gặp hiện tượng này, hướng xử lý hợp lý là thêm biến đa thức (PolynomialFeatures), biến tương tác, hoặc chuyển sang mô hình phi tuyến (Random Forest, Gradient Boosting) như đã đề xuất ở mục 43, thay vì tiếp tục ép quan hệ đó vào một đường thẳng.

Câu 8. Bài tập mở rộng (khuyến khích)

```
In [73]: from sklearn.ensemble import RandomForestRegressor
import time

rf_model = RandomForestRegressor(random_state=42, n_jobs=-1)

start = time.time()
rf_model.fit(X_train, y_train)
rf_train_time = time.time() - start

y_train_pred_rf = rf_model.predict(X_train)
y_test_pred_rf = rf_model.predict(X_test)

mae_train_rf = mean_absolute_error(y_train, y_train_pred_rf)
rmse_train_rf = np.sqrt(mean_squared_error(y_train, y_train_pred_rf))
r2_train_rf = r2_score(y_train, y_train_pred_rf)

mae_test_rf = mean_absolute_error(y_test, y_test_pred_rf)
rmse_test_rf = np.sqrt(mean_squared_error(y_test, y_test_pred_rf))
r2_test_rf = r2_score(y_test, y_test_pred_rf)

print(f"Thời gian huấn luyện RandomForest: {rf_train_time:.2f} giây")
print("Train - MAE:", mae_train_rf, "RMSE:", rmse_train_rf, "R2:", r2_train_rf)
print("Test - MAE:", mae_test_rf, "RMSE:", rmse_test_rf, "R2:", r2_test_rf)
```

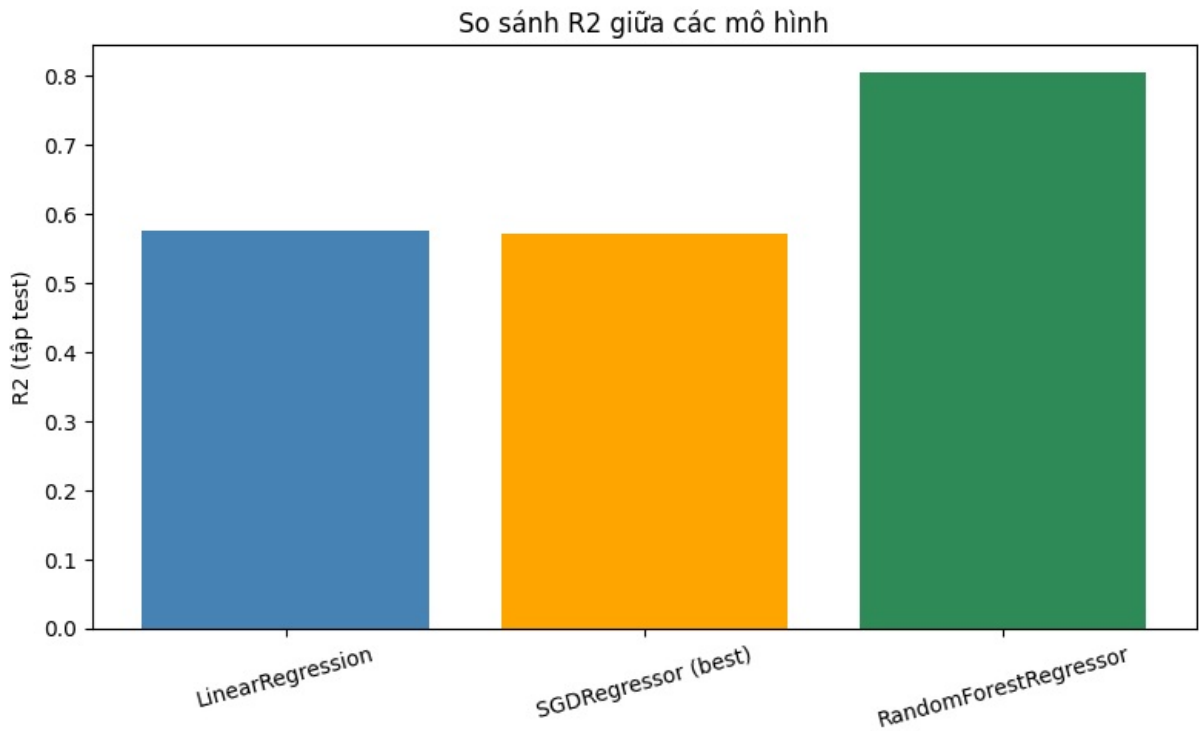
Thời gian huấn luyện RandomForest: 15.71 giây
Train - MAE: 0.12205671543120196 RMSE: 0.18795619089802199 R2: 0.9735726320302575
Test - MAE: 0.32754256845930246 RMSE: 0.5053399773665033 R2: 0.8051230593157366

```
In [74]: final_comparison_df = pd.DataFrame({
    'Model': ['LinearRegression', 'SGDRegressor (best)', 'RandomForestRegressor'],
    'MAE': [mae_test, best_config['MAE'], mae_test_rf],
    'RMSE': [rmse_test, best_config['RMSE'], rmse_test_rf],
    'R2': [r2_test, best_config['R2'], r2_test_rf]
}).round(4)

print(final_comparison_df)
```

	Model	MAE	RMSE	R2
0	LinearRegression	0.5332	0.7456	0.5758
1	SGDRegressor (best)	0.5328	0.7498	0.5710
2	RandomForestRegressor	0.3275	0.5053	0.8051

```
In [75]: plt.figure(figsize=(8, 5))
plt.bar(final_comparison_df['Model'], final_comparison_df['R2'], color=['steelblue', 'orange', 'seagreen'])
plt.ylabel('R2 (tập test)')
plt.title('So sánh R2 giữa các mô hình')
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```



Nhận xét biểu đồ: RandomForestRegressor vượt trội rõ rệt so với hai mô hình tuyến tính, đạt $R^2 = 0.8051$ trên tập test — cao hơn khoảng 0.23 điểm (tương đương ~40%) so với LinearRegression (0.5758) và SGDRegressor (0.5710), trong khi hai mô hình tuyến tính gần như ngang bằng nhau. Sự chênh lệch lớn này cho thấy phần lớn biến thiên giá nhà mà LinearRegression bỏ lỡ đến từ các quan hệ phi tuyến và tương tác giữa feature (như vị trí địa lý, tuổi nhà) mà RandomForest nắm bắt được nhờ cấu trúc cây quyết định.

```
In [76]: print(f"RandomForest - R2 train: {r2_train_rf:.4f}, R2 test: {r2_test_rf:.4f}")
print(f"Chênh lệch train-test: {r2_train_rf - r2_test_rf:.4f}")
```

```
RandomForest - R2 train: 0.9736, R2 test: 0.8051
Chênh lệch train-test: 0.1684
```

Kết luận

- Kết quả thực nghiệm cho thấy RandomForestRegressor cải thiện đáng kể so với hai mô hình tuyến tính: R^2 test tăng từ 0.5758 (LinearRegression) lên 0.8051 — tương đương cải thiện khoảng 0.23 điểm (~40%); RMSE giảm từ 0.7456 xuống 0.5053 (giảm ~32%); MAE giảm từ 0.5332 xuống 0.3275 (giảm ~39%, tương đương sai số trung bình giảm từ ~\$53,320 xuống ~\$32,750). Mức cải thiện này xác nhận đúng giả thuyết đã nêu ở mục 41 và 43: giới hạn chính của các mô hình tuyến tính không nằm ở phương pháp tối ưu (SGDRegressor dù được tinh chỉnh tốt cũng không vượt qua LinearRegression) mà nằm ở giả định tuyến tính — RandomForest với cấu trúc cây quyết định nắm bắt được các quan hệ phi tuyến và tương tác giữa feature, ví dụ hiệu ứng cụm địa lý (giá nhà cao tập trung ở Bay Area/LA bất kể MedInc trung bình) và quan hệ không đơn điệu giữa HouseAge và giá nhà, những điều mà một đường thẳng hồi quy không thể biểu diễn.
- Tuy nhiên, sự cải thiện này đi kèm với đánh đổi rõ rệt. Về chi phí tính toán: RandomForest mất 15.71 giây để huấn luyện — chậm hơn nhiều lần so với LinearRegression (giải bằng closed-form gần như tức thời), do phải xây dựng hàng trăm cây quyết định và duyệt qua dữ liệu lặp lại nhiều lần; chi phí này sẽ tăng nhanh hơn nếu dữ liệu lớn hơn (như tình huống đã bàn ở mục 47). Về dấu hiệu overfitting: R^2 train = 0.9736 so với R^2 test = 0.8051, chênh lệch 0.1684 — khoảng cách này lớn hơn nhiều so với LinearRegression (vốn gần như không overfitting), cho thấy RandomForest có xu hướng khớp rất sát với dữ liệu train và cần được kiểm soát thêm (ví dụ giới hạn max_depth, tăng min_samples_leaf, hoặc dùng cross-validation) để đảm bảo độ ổn định khi triển khai thực tế. Về khả năng diễn giải: LinearRegression cho phép đọc trực tiếp hệ số để biết chiều và mức độ ảnh hưởng của từng feature, trong khi RandomForest chỉ cung cấp feature_importances_ (mức độ quan trọng tổng thể, không có dấu +/-) và mang tính "hộp đen" hơn, khó giải thích một dự đoán cụ thể nếu không dùng thêm công cụ như SHAP.

- Tóm lại, việc lựa chọn mô hình cuối cùng nên dựa trên mục tiêu bài toán: nếu ưu tiên độ chính xác dự đoán, RandomForest (hoặc Gradient Boosting) là lựa chọn tốt hơn rõ rệt; nếu ưu tiên tốc độ huấn luyện, chi phí tính toán thấp và khả năng diễn giải minh bạch từng hệ số, LinearRegression vẫn có giá trị riêng dù độ chính xác thấp hơn. Một hướng tiếp theo hợp lý là kiểm soát overfitting của RandomForest bằng cross-validation và tinh chỉnh siêu tham số, đồng thời thử nghiệm GradientBoostingRegressor để so sánh thêm.

In []: